

LAPORAN PRAKTIKUM
Arsitektur Monolith vs Microservices dengan Flask Python



Disusun oleh :
Nama : FAJRI AYU
NIM : 2024903430049

TEKNOLOGI INFORMASI DAN KOMPUTER
TEKNOLOGI REKAYASA KOMPUTER JARINGAN
POLITEKNIK NEGERI LHOKSEUMAWE
TAHUN 2026

1. Tujuan Pembelajaran

- Memahami konsep dasar aplikasi Monolith dan Microservices.
- Mampu membangun backend sederhana menggunakan Flask Python.
- Menganalisis kelebihan dan kekurangan dari masing-masing arsitektur melalui praktik langsung.

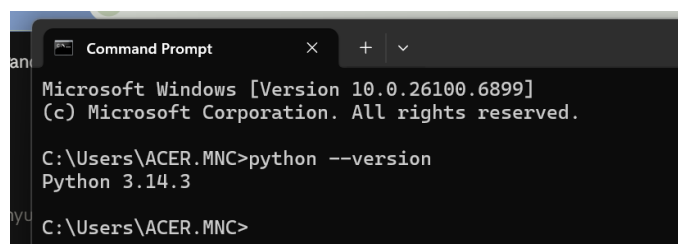
2. Konsep Dasar

Monolith: seluruh fitur aplikasi (katalog buku, pembayaran, pesanan) disatukan dalam satu codebase dan berjalan di satu server/proses yang sama.

Microservices: fitur aplikasi dipecah menjadi layanan-layanan kecil yang mandiri (Book Service dan Order Service). Setiap layanan berjalan di server/port masing-masing dan saling berkomunikasi via HTTP/API.

3. Persiapan Lingkungan

Sebelum memulai, dilakukan pengecekan versi Python yang terinstal, lalu instalasi library Flask dan requests menggunakan pip.



```
Microsoft Windows [Version 10.0.26100.6899]
(c) Microsoft Corporation. All rights reserved.

C:\Users\ACER.MNC>python --version
Python 3.14.3

C:\Users\ACER.MNC>
```

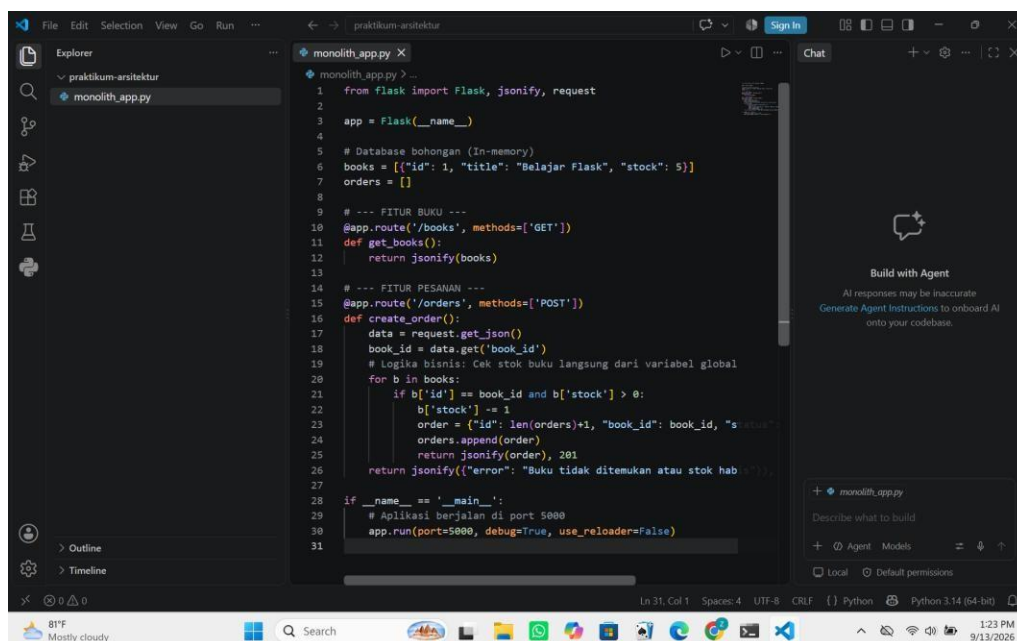
Gambar 1. Verifikasi instalasi Python (versi 3.14.3)

Catatan teknis: karena versi Python yang digunakan (3.14) tergolong sangat baru, versi Flask lama yang awalnya terinstal (2.2.5) mengalami error kompatibilitas (AttributeError: module 'pkgutil' has no attribute 'get_loader'). Masalah ini diselesaikan dengan meng-upgrade Flask ke versi terbaru (3.1.3) menggunakan perintah pip install --upgrade Flask.

4. Bagian 1: Membangun Aplikasi Monolith

4.1 Kode Program

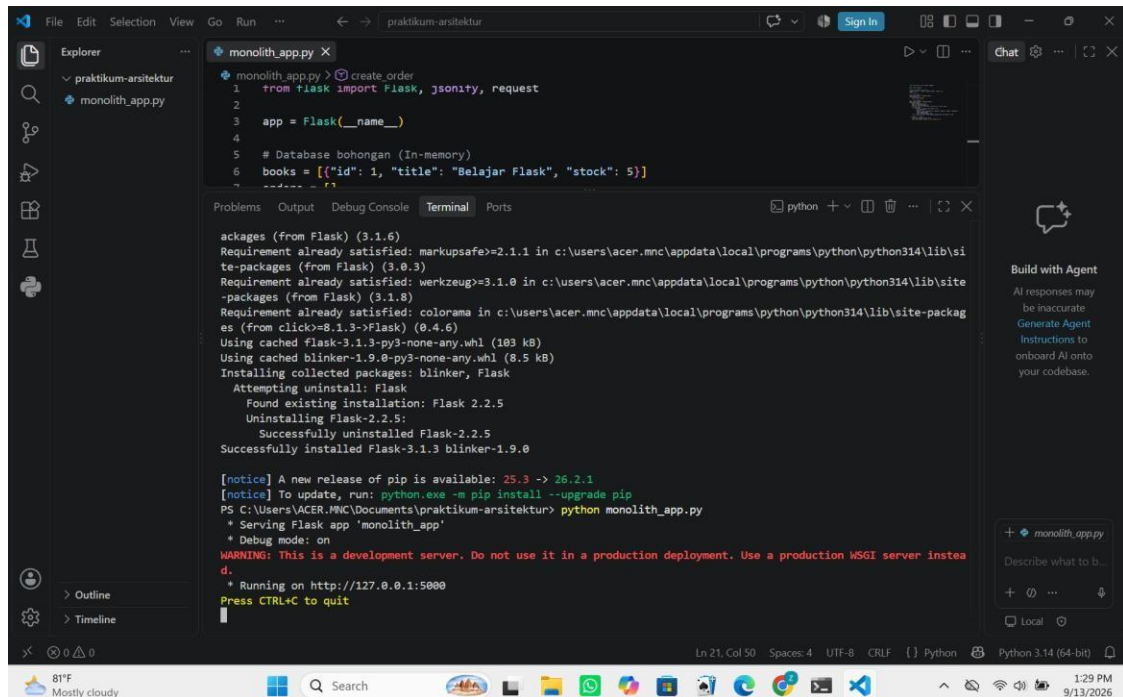
File monolith_app.py dibuat berisi seluruh logika Buku dan Pesanan dalam satu aplikasi Flask yang berjalan di port 5000.



```
1 from flask import Flask, jsonify, request
2
3 app = Flask(__name__)
4
5 # Database bohongan (In-memory)
6 books = [{"id": 1, "title": "Belajar Flask", "stock": 5}]
7 orders = []
8
9 # --- FITUR BUKU ---
10 @app.route('/books', methods=['GET'])
11 def get_books():
12     return jsonify(books)
13
14 # --- FITUR PESANAN ---
15 @app.route('/orders', methods=['POST'])
16 def create_order():
17     data = request.get_json()
18     book_id = data.get('book_id')
19     # Logika bisnis: Cek stok buku langsung dari variabel global
20     for b in books:
21         if b['id'] == book_id and b['stock'] > 0:
22             b['stock'] -= 1
23             order = {"id": len(orders)+1, "book_id": book_id, "stock": b['stock']}
24             orders.append(order)
25             return jsonify(order), 201
26     return jsonify({"error": "Buku tidak ditemukan atau stok habis"}), 400
27
28 if __name__ == '__main__':
29     # Aplikasi berjalan di port 5000
30     app.run(port=5000, debug=True, use_reloader=False)
```

Gambar 2. Kode program monolith_app.py di VS Code

4.2 Menjalankan Aplikasi



Gambar 3. Aplikasi monolith berhasil dijalankan (Running on <http://127.0.0.1:5000>)

4.3 Pengujian Endpoint

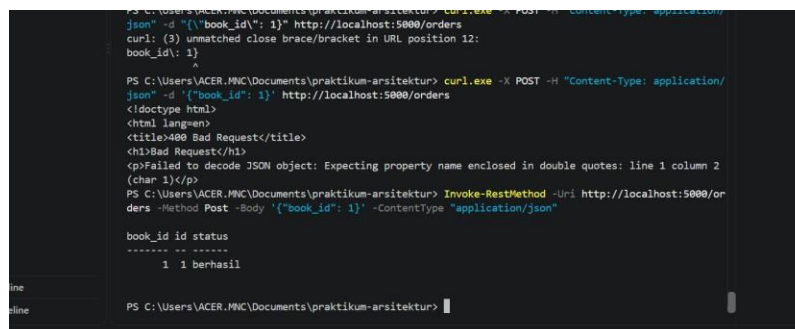
Pengujian dilakukan menggunakan curl.exe dan Invoke-RestMethod pada terminal PowerShell.

```
curl.exe http://localhost:5000/books
```

```
Invoke-RestMethod -Uri http://localhost:5000/orders -Method Post `
-Body '{"book_id": 1}' -ContentType "application/json"
```



Gambar 4. Hasil GET /books — stok awal buku "Belajar Flask" = 5



Gambar 5. Hasil POST /orders — pesanan berhasil dibuat (status: berhasil)

```
PS C:\Users\ACER.MNC\Documents\praktikum-arsitektur> curl.exe http://localhost:5000/books
[
  {
    "id": 1,
    "stock": 4,
    "title": "Belajar Flask"
  }
]
PS C:\Users\ACER.MNC\Documents\praktikum-arsitektur>
```

Gambar 6. Hasil GET /books setelah pesanan — stok berkurang menjadi 4

Hasil pengujian menunjukkan bahwa fitur Buku dan Pesanan pada arsitektur Monolith bekerja dengan benar: setiap pesanan baru langsung mengurangi stok buku karena kedua fitur berbagi variabel data yang sama dalam satu proses aplikasi.

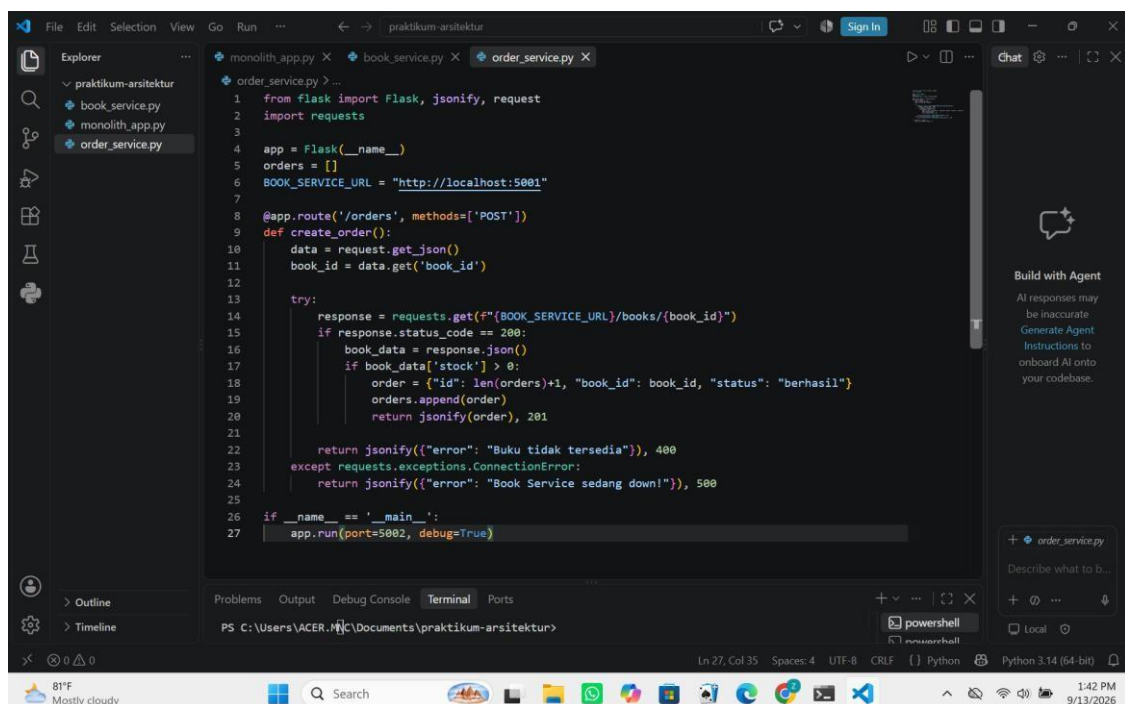
5. Bagian 2: Membangun Aplikasi Microservices

Sistem yang sama dipecah menjadi dua layanan independen: Book Service (port 5001) yang mengelola data buku, dan Order Service (port 5002) yang mengelola pesanan dan berkomunikasi ke Book Service melalui HTTP request.

5.1 Kode Program

book_service.py — melayani data buku secara mandiri di port 5001 (kode identik dengan modul praktikum, ditampilkan pada Bagian 4.1 untuk struktur, isi lengkap tersedia pada lampiran kode program).

order_service.py — menerima permintaan pesanan, lalu memanggil Book Service melalui HTTP request untuk memvalidasi stok sebelum menyimpan pesanan.

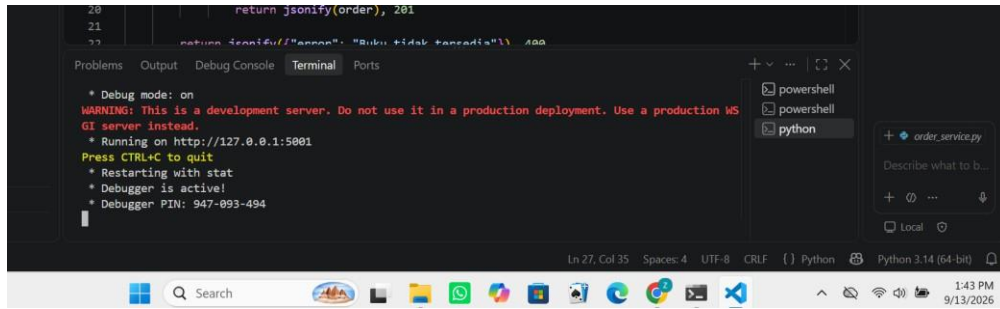


```
1 from flask import Flask, jsonify, request
2 import requests
3
4 app = Flask(__name__)
5 orders = []
6 BOOK_SERVICE_URL = "http://localhost:5001"
7
8 @app.route('/orders', methods=['POST'])
9 def create_order():
10     data = request.get_json()
11     book_id = data.get('book_id')
12
13     try:
14         response = requests.get(f"{BOOK_SERVICE_URL}/books/{book_id}")
15         if response.status_code == 200:
16             book_data = response.json()
17             if book_data['stock'] > 0:
18                 order = {"id": len(orders)+1, "book_id": book_id, "status": "berhasil"}
19                 orders.append(order)
20                 return jsonify(order), 201
21
22         return jsonify({"error": "Buku tidak tersedia"}), 400
23     except requests.exceptions.ConnectionError:
24         return jsonify({"error": "Book Service sedang down!"}), 500
25
26 if __name__ == '__main__':
27     app.run(port=5002, debug=True)
```

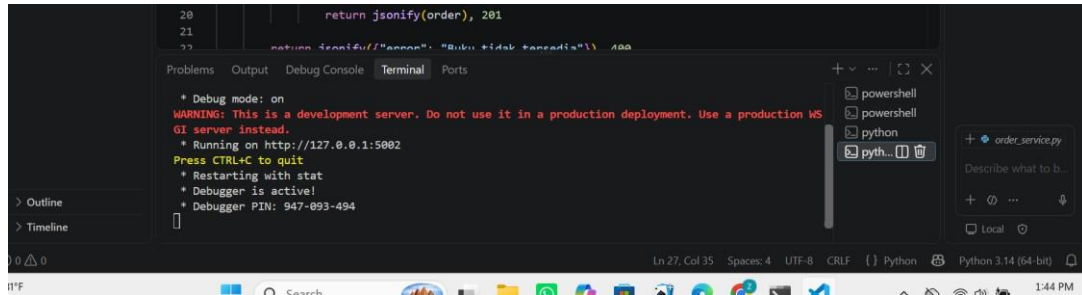
Gambar 7. Kode program order_service.py di VS Code

5.2 Menjalankan Kedua Layanan

Kedua service dijalankan secara bersamaan pada terminal terpisah agar dapat saling berkomunikasi.



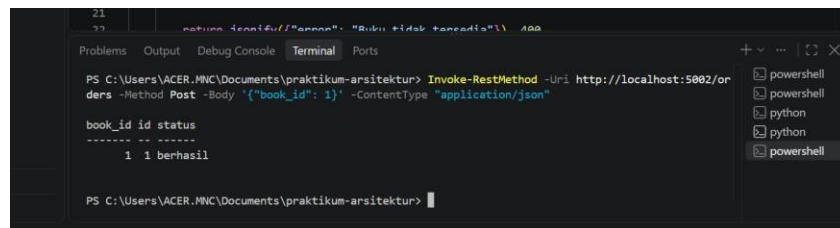
Gambar 8. Book Service berjalan di port 5001



Gambar 9. Order Service berjalan di port 5002

5.3 Pengujian Komunikasi Antar-Service

```
Invoke-RestMethod -Uri http://localhost:5002/orders -Method Post `
-Body '{"book_id": 1}' -ContentType "application/json"
```

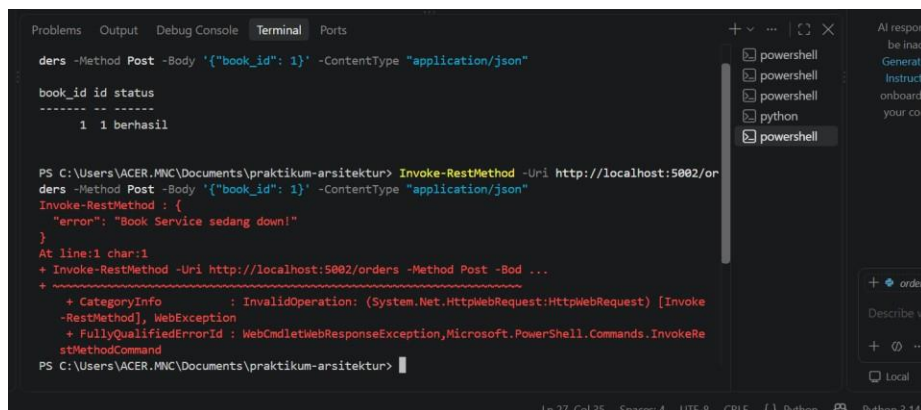


Gambar 10. Order Service (5002) berhasil memvalidasi stok ke Book Service (5001) dan membuat pesanan

Pengujian ini membuktikan bahwa Order Service tidak lagi mengakses data buku secara langsung, melainkan harus melakukan HTTP request ke Book Service — sesuai dengan prinsip dasar arsitektur microservices.

6. Eksperimen Kegagalan (Fault Isolation)

Book Service dimatikan secara paksa (Ctrl+C), kemudian dikirim ulang permintaan pesanan ke Order Service untuk mengamati perilaku sistem saat salah satu layanan mengalami gangguan.



Gambar 11. Order Service tetap merespons dengan pesan error terkontrol saat Book Service down

Hasil eksperimen menunjukkan bahwa Order Service tidak ikut mati atau crash total. Ia tetap berjalan dan merespons dengan pesan error yang jelas: "Book Service sedang down!" (HTTP 500). Inilah yang disebut Fault Isolation — kegagalan pada satu layanan tidak menjalar dan melumpuhkan layanan lain dalam sistem.

7. Pembahasan / Bahan Diskusi

7.1 Crash pada Fitur Pesanan: Monolith vs Microservices

Pada arsitektur Monolith, apabila fitur Pesanan mengalami crash atau bug fatal yang mematikan proses, seluruh aplikasi—termasuk fitur Buku—akan ikut down, karena keduanya berjalan dalam satu proses/server yang sama. Sebaliknya, pada arsitektur Microservices, karena Order Service dan Book Service berjalan sebagai proses terpisah di port berbeda, kegagalan pada Order Service tidak akan mematikan Book Service. Pengguna tetap dapat melihat katalog buku meskipun fitur pesanan sedang bermasalah. Prinsip inilah yang disebut Fault Isolation, dan telah dibuktikan langsung pada eksperimen di Bagian 6.

7.2 Solusi Mengatasi Latensi Jaringan

Beberapa pendekatan yang umum digunakan di industri untuk mengatasi latensi akibat komunikasi HTTP antar-service:

- Caching — menyimpan hasil respons Book Service sementara (misalnya menggunakan Redis) agar tidak perlu memanggil ulang untuk data yang jarang berubah.
- Asynchronous / Event-driven communication — menggunakan message broker seperti RabbitMQ atau Kafka agar antar-service tidak perlu saling menunggu respons secara langsung (blocking).
- Circuit breaker pattern — menghentikan sementara pemanggilan ke service yang sering gagal atau lambat, agar tidak membebani sistem lebih lanjut.
- Connection pooling & timeout tuning — membuat request HTTP antar-service lebih efisien dan mencegah request menggantung terlalu lama.
- Data replication / local cache per service — menyimpan salinan data yang dibutuhkan langsung di service pemanggil, dengan trade-off terhadap konsistensi data.

Antar-Service

7.3 Strategi Scaling saat Traffic Pencarian Buku Melonjak

Apabila traffic pencarian buku melonjak drastis sementara traffic pesanan biasa saja, maka layanan yang perlu di-scale-up adalah Book Service (port 5001), misalnya dengan menjalankan beberapa instance Book Service di belakang load balancer. Order Service (port 5002) tidak perlu ditambah kapasitas. Inilah keunggulan arsitektur microservices dibandingkan monolith: setiap fitur dapat di-scale secara independen sesuai kebutuhan. Pada arsitektur monolith, karena seluruh fitur berada dalam satu proses, scaling harus dilakukan terhadap seluruh aplikasi sekaligus meskipun hanya satu fitur yang membutuhkan kapasitas lebih besar, sehingga penggunaan resource menjadi kurang efisien.

8. Kesimpulan

Melalui praktik langsung, dapat disimpulkan bahwa:

- Arsitektur Monolith lebih sederhana dibangun dan cocok untuk aplikasi berskala kecil, namun memiliki risiko satu titik kegagalan (single point of failure) yang dapat melumpuhkan seluruh sistem.
- Arsitektur Microservices menawarkan isolasi kegagalan (fault isolation) dan fleksibilitas scaling per-fitur, namun menambah kompleksitas berupa komunikasi jaringan antar-service dan potensi latensi.
- Pemilihan arsitektur yang tepat bergantung pada skala aplikasi, ukuran tim pengembang, dan kebutuhan bisnis terhadap ketahanan (resilience) serta kemampuan scaling sistem.